A Journey into Windows Security Support Provider Interface

Yuki Chen (@guhe120)

Who am I

- Independent Security Researcher
- Pwn2Own 15/16/17, Tianfu Cup 18/19/20
- 5 times MSRC most valuable researcher yearly TOP 1
- 2 Pwnie Awards – Best RCE and Epic Achievement
- Japanese-Language Proficiency Test N1
- Hardcore ACG Otaku

- CVE-2014-0290,CVE-2014-0321,CVE-2014-1753,CVE-2014-1769,CVE-2014-1782,CVE-2014-1804,CVE-2014-2768,CVE-2014-2802,CVE-2014-2803,CVE-2014-2824,CVE-2014-4057,CVE-2014-4092,CVE-2014-4091, CVE-2014-4095,CVE-2014-4096,CVE-2014-4097,CVE-2014-4082,CVE-2014-4105,CVE-2014-4129,CVE-2014-6369,CVE-2015-0029,CVE-2015-1745,CVE-2015-1743,CVE-2015-3134,CVE-2015-3135,CVE-2015-4431, CVE-2015-5552,CVE-2015-5553,CVE-2015-5559,CVE-2015-6682,CVE-2015-7635,CVE-2015-7636,CVE-2015-7637,CVE-2015-7638,CVE-2015-7639,CVE-2015-7640,CVE-2015-7641,CVE-2015-7642,CVE-2015-7643, CVE-2015-8454,CVE-2015-8059,CVE-2015-8058,CVE-2015-8055,CVE-2015-8057,CVE-2015-8056,CVE-2015-8061,CVE-2015-8067,CVE-2015-8066,CVE-2015-8062,CVE-2015-8068,CVE-2015-8064,CVE-2015-8065, CVE-2015-8063,CVE-2015-8405,CVE-2015-8404,CVE-2015-8402,CVE-2015-8403,CVE-2015-8071,CVE-2015-8401,CVE-2015-8406,CVE-2015-8069,CVE-2015-8070,CVE-2015-8440,CVE-2015-8409,CVE-2015-8047, CVE-2015-8455,CVE-2015-8045,CVE-2015-8441,CVE-2016-0980,CVE-2016-1015,CVE-2016-1016,CVE-2016-1017,CVE-2016-4120,CVE-2016-4160,CVE-2016-4161,CVE-2016-4162,CVE-2016-4163,CVE-2016-4185, CVE-2016-4249,CVE-2016-4180,CVE-2016-4181,CVE-2016-4183,CVE-2016-4184,CVE-2016-4186,CVE-2016-4187,CVE-2016-4233,CVE-2016-4234,CVE-2016-4235,CVE-2016-4236,CVE-2016-4237,CVE-2016-4238, CVE-2016-4239,CVE-2016-4240,CVE-2016-4241,CVE-2016-4242, CVE-2016-4243,CVE-2016-4244,CVE-2016-4245,CVE-2016-4246,CVE-2016-4182,CVE-2016-3375,CVE-2017-3001,CVE-2017-3002,CVE-2017-3003, CVE-2017-0238,CVE-2017-0236,CVE-2017-8549,CVE-2017-8619,CVE-2017-11887,CVE-2017-11913,CVE-2017-11846,CVE-2017-8753CVE-2018-8618,CVE-2018-8544,CVE-2018-8367,CVE-2018-8372, CVE-2018-8242,CVE-2018-8236,CVE-2018-1022,CVE-2018-0951,CVE-2018-0953,CVE-2018-8122,CVE-2018-8114,CVE-2018-0955,CVE-2018-0954,CVE-2018-1000,CVE-2018-0981,CVE-2018-0988,CVE-2018-0997, CVE-2018-1004,CVE-2018-0994,CVE-2018-0872,CVE-2018-0866,CVE-2018-0834,CVE-2018-0798,CVE-2018-0807,CVE-2018-0845,CVE-2018-0802,CVE-2018-0806,CVE-2018-0805,CVE-2018-0849,CVE-2018-0848, CVE-2018-0862,CVE-2018-0804,CVE-2018-0801,CVE-2018-0812,CVE-2019-1485,CVE-2019-1484,CVE-2019-1390,CVE-2019-1239,CVE-2019-1238,CVE-2019-1060,CVE-2019-1237,CVE-2019-1236,CVE-2019-1221, CVE-2019-1057,CVE-2019-1004,CVE-2019-1001,CVE-2019-1059,CVE-2019-1056,CVE-2019-0988,CVE-2019-1055,CVE-2019-1005,CVE-2019-0810,CVE-2019-0862,CVE-2019-0842,CVE-2019-0806,CVE-2019-0795,CVE-2019-0794,CVE-2019-0793,CVE-2019-0792,CVE-2019-0791,CVE-2019-0790,CVE-2019-0753,CVE-2019-0667,CVE-2019-0592,CVE-2019-0773,CVE-2019-0772,CVE-2019-0639,CVE-2019-0665,CVE-2019-0611,CVE-2019-0680,CVE-2019-0756,CVE-2019-0666,CVE-2019-0651,CVE-2019-0591,CVE-2019-0655,CVE-2019-0610,CVE-2019-0606,CVE-2019-0605,CVE-2019-0652,CVE-2019-0567,CVE-2020-16964,CVE-2020-16962,CVE-2020-16961,CVE-2020-16960,CVE-2020-16959,CVE-2020-16958,CVE-2020-16963,CVE-2020-17055,CVE-2020-17044,CVE-2020-17034,CVE-2020-17028,CVE-2020-17027,CVE-2020-17025,CVE-2020-17033,CVE-2020-17032,CVE-2020-17031,CVE-2020-17026,CVE-2020-16974,CVE-2020-16975,CVE-2020-16972,CVE-2020-16905,CVE-2020-16876,CVE-2020-16976,CVE-2020-16973,CVE-2020-16936,CVE-2020-16912,CVE-2020-16887,CVE-2020-1133,CVE-2020-1052,CVE-2020-0912,CVE-2020-1598,CVE-2020-1491,CVE-2020-1376,CVE-2020-1130,CVE-2020-0922,CVE-2020-1579,CVE-2020-1567,CVE-2020-1551,CVE-2020-1547,CVE-2020-1536,CVE-2020-1539,CVE-2020-1535,CVE-2020-1534,CVE-2020-1519,CVE-2020-1546,CVE-2020-1545,CVE-2020-1544,CVE-2020-1543,CVE-2020-1542,CVE-2020-1541,CVE-2020-1540,CVE-2020-1428,CVE-2020-1427,CVE-2020-1430,CVE-2020-1411,CVE-2020-1390,CVE-2020-1373,CVE-2020-1365,CVE-2020-1336,CVE-2020-1437,CVE-2020-1406,CVE-2020-1403,CVE-2020-1388,CVE-2020-1371,CVE-2020-1354,CVE-2020-1085,CVE-2020-1257,CVE-2020-1255,CVE-2020-1197,CVE-2020-1278,CVE-2020-1271,CVE-2020-1260,CVE-2020-1216,CVE-2020-1214,CVE-2020-1212,CVE-2020-1234,CVE-2020-1293,CVE-2020-1281,CVE-2020-1230,CVE-2020-1215,CVE-2020-1203,CVE-2020-1202,CVE-2020-1132,CVE-2020-1064,CVE-2020-1058,CVE-2020-1056,CVE-2020-1093,CVE-2020-1060,CVE-2020-1067,CVE-2020-1035,CVE-2020-1021,CVE-2020-1061,CVE-2020-1015,CVE-2020-0967,CVE-2020-0966,CVE-2020-0895,CVE-2020-0833,CVE-2020-0832,CVE-2020-0831,CVE-2020-0824,CVE-2020-0847,CVE-2020-0640,CVE-2021-40469,CVE-2021-40467,CVE-2021-40466,CVE-2021-26435,CVE-2021-36963,CVE-2021-34534,CVE-2021-33746,CVE-2021-34525,CVE-2021-34442,CVE-2021-34497,CVE-2021-34447,CVE-2021-34494,CVE-2021-33780,CVE-2021-33756,CVE-2021-33754,CVE-2021-33752,CVE-2021-33750,CVE-2021-33749,CVE-2021-31194,CVE-2021-28434,CVE-2021-28358,CVE-2021-28357,CVE-2021-28356,CVE-2021-28355,CVE-2021-28354,CVE-2021-28353,CVE-2021-28352,CVE-2021-28346,CVE-2021-28345,CVE-2021-28344,CVE-2021-28343,CVE-2021-28342,CVE-2021-28341,CVE-2021-28340,CVE-2021-28339,CVE-2021-28338,CVE-2021-28337,CVE-2021-28336,CVE-2021-28335,CVE-2021-28334,CVE-2021-28333,CVE-2021-28332,CVE-2021-28331,CVE-2021-28330,CVE-2021-28329,CVE-2021-27088,CVE-2021-26901,CVE-2021-26899,CVE-2021-26898,CVE-2021-26872,CVE-2021-24103,CVE-2021-24102,CVE-2021-1673,CVE-2021-1667,CVE-2021-1658,CVE-2021-1649,CVE-2021-1702,CVE-2021-1701,CVE-2021-1700,CVE-2021-1671,CVE-2021-1666,CVE-2021-1664,CVE-2021-1660,CVE-2022-44670,CVE-2022-44676,CVE-2022-41039,CVE-2022-41088,CVE-2022-41081,CVE-2022-30198,CVE-2022-22035,CVE-2022-33634,CVE-2022-24504,CVE-2022-38000,CVE-2022-38048,CVE-2022-38047,CVE-2022-35830,CVE-2022-34722,CVE-2022-34721,CVE-2022-35794,CVE-2022-35767,CVE-2022-35766,CVE-2022-35753,CVE-2022-35752,CVE-2022-35745,CVE-2022-35744,CVE-2022-34714,CVE-2022-34702,CVE-2022-22039,CVE-2022-22038,CVE-2022-22029,CVE-2022-30161,CVE-2022-30153,CVE-2022-30149,CVE-2022-30146,CVE-2022-30143,CVE-2022-30142,CVE-2022-30141,CVE-2022-30139,CVE-2022-30136,CVE-2022-29141,CVE-2022-29139,CVE-2022-29137,CVE-2022-29131,CVE-2022-29130,CVE-2022-29129,CVE-2022-29128,CVE-2022-22014,CVE-2022-22013,CVE-2022-22012,CVE-2022-26937,CVE-2022-21972,CVE-2022-23270,CVE-2022-26919,CVE-2022-26825,CVE-2022-26824,CVE-2022-26823,CVE-2022-26822,CVE-2022-26821,CVE-2022-26820,CVE-2022-26819,CVE-2022-26818,CVE-2022-26817,CVE-2022-26815,CVE-2022-26814,CVE-2022-26813,CVE-2022-26812,CVE-2022-26811,CVE-2022-24500,CVE-2022-24497,CVE-2022-24541,CVE-2022-24492,CVE-2022-24534,CVE-2022-24485,CVE-2022-24528,CVE-2022-21983,CVE-2023-24903,CVE-2023-28283,CVE-2023-28240,CVE-2023-28238,CVE-2023-28220,CVE-2023-28219,CVE-2023-23404,CVE-2023-23414,CVE-2023-23407,CVE-2023-23385,CVE-2023-21692,CVE-2023-21712,CVE-2023-21679,CVE-2023-21556,CVE-2023-21555,CVE-2023-21548,CVE-2023-21546,CVE-2023-21535, CVE-2023-21681,CVE-2023-21799,CVE-2023-21704,CVE-2023-21718,CVE-2023-23384,CVE-2023-28275,CVE-2023-29372,CVE-2023-29356,CVE-2023-32025,CVE-2023-32026,CVE-2023-32027,CVE-2023-36577,CVE-2023-36402,CVE-2023-36006,CVE-2024-0056,CVE-2024-21367,CVE-2024-21369,CVE-2024-21391,CVE-2024-21350,CVE-2024-21352,CVE-2024-21358,CVE-2024-21359,CVE-2024-21360,CVE-2024-21361,CVE-2024-21365,CVE-2024-21366,CVE-2024-21368,CVE-2024-21370,CVE-2024-21375,CVE-2024-21420,CVE-2024-21444,CVE-2024-21450,CVE-2024-26166,CVE-2024-21441,CVE-2024-26161,CVE-2024-28927,CVE-2024-28937,CVE-2024-28940,CVE-2024-28941,CVE-2024-28943,CVE-2024-28944,CVE-2024-28945,CVE-2024-29985,CVE-2024-29047,CVE-2024-26244,CVE-2024-26214,CVE-2024-28906,CVE-2024-28908,CVE-2024-28909,CVE-2024-28910,CVE-2024-28911,CVE-2024-28912,CVE-2024-28913,CVE-2024-28914,CVE-2024-28915,CVE-2024-28926,CVE-2024-28929,CVE-2024-28930,CVE-2024-28931,CVE-2024-28932,CVE-2024-28933,CVE-2024-28934,CVE-2024-28935,CVE-2024-28936,CVE-2024-28938,CVE-2024-28939,CVE-2024-28942,CVE-2024-29043,CVE-2024-29044,CVE-2024-29045,CVE-2024-29046,CVE-2024-29048,CVE-2024-29982,CVE-2024-29983,CVE-2024-29984,CVE-2024-30006,CVE-2024-38088,CVE-2024-38087,CVE-2024-21332,CVE-2024-21333,CVE-2024-21335,CVE-2024-21373,CVE-2024-21398,CVE-2024-21414,CVE-2024-21415,CVE-2024-21428,CVE-2024-37318,CVE-32024-37332,CVE-2024-37331,CVE-2024-35271,CVE-2024-35272,CVE-2024-20701,CVE-2024-21303,CVE-2024-21308,CVE-2024-21317,CVE-2024-21331,CVE-2024-21425,CVE-2024-37319,CVE-2024-37320,CVE-2024-37321,CVE-2024-37322,CVE-2024-37323,CVE-2024-37324,CVE-2024-21449,CVE-2024-37326,CVE-2024-37327,CVE-2024-37328,CVE-2024-37329,CVE-2024-37330,CVE-2024-37334,CVE-2024-37333,CVE-2024-37336,CVE-2024-28928,CVE-2024-35256, CVE-2024-49112,CVE-2024-49113,CVE-2024-49121,CVE-2024-49126,CVE-2024-49127,CVE-2025-21294,CVE-2025-21295,CVE-2025-21351,CVE-2025-21368,CVE-2025-21369

My daily life as ~~Independent Security Researcher~~ OTAKU



Agenda

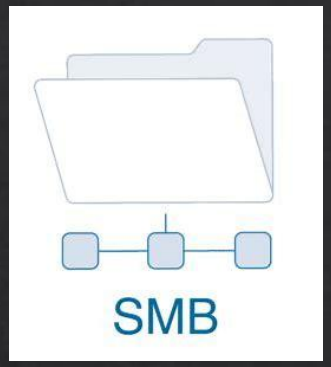
- Background
- Windows SSPI Architecture
- Windows SSPI Bugs
- Conclusion

Background

Windows Security Support Provider Interface (SSPI)

- SSPI: a universal, industry-standard interface for secure distributed applications
  - Authentication
  - Encryption/Decryption
  - Secure Data Schannel















Authentication

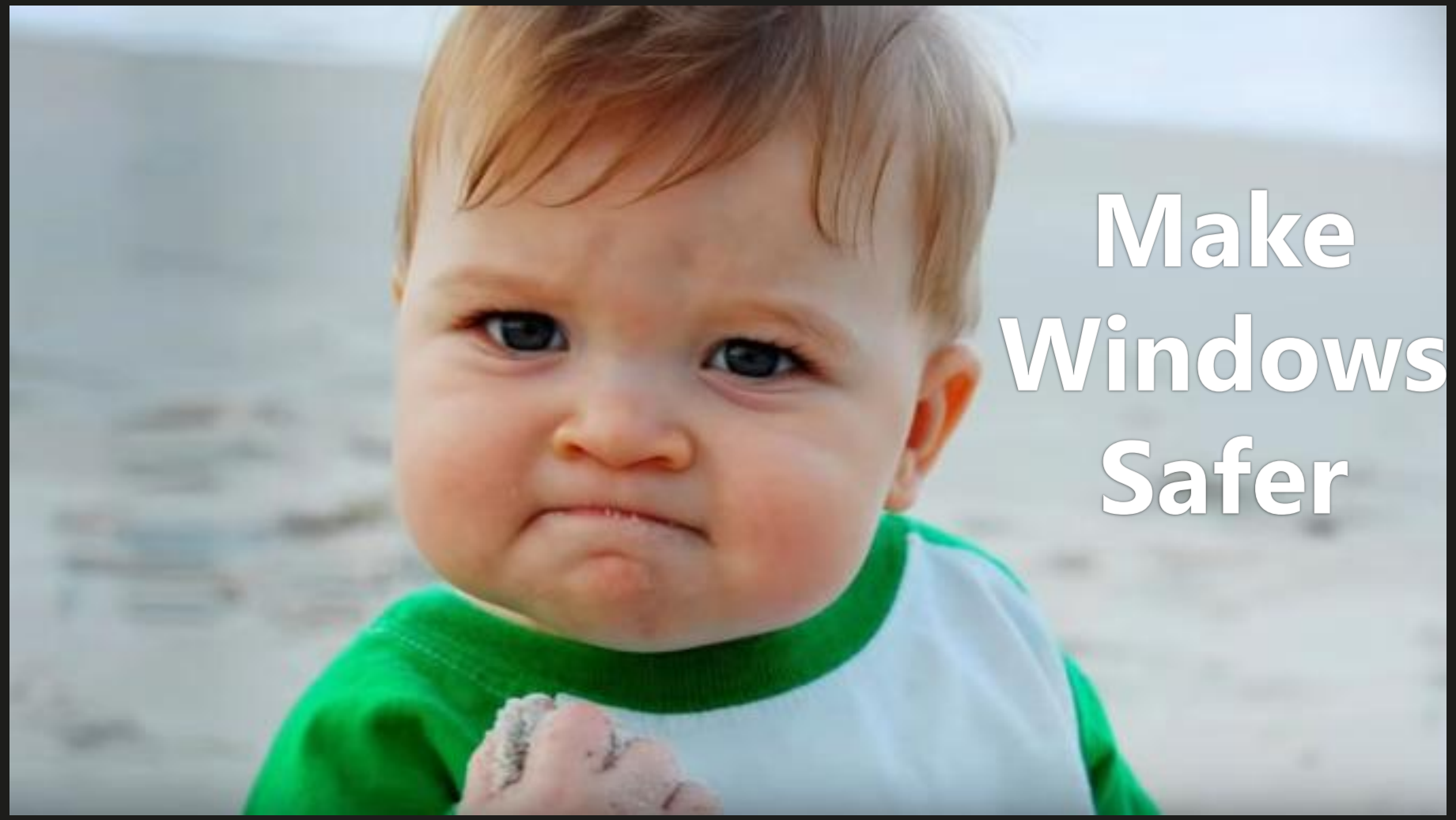
NTLM
Kerberos
Negotiation / Negoexts
Digest
Schannel
CredSSP

SSPI

Why SSPI Research

- The very front defense line of Windows security
  - Many ways to trigger
  - Could have big local/remote security impact
- Not much comprehensive research previously
- Seems to be a fun & challenging task

Motivation



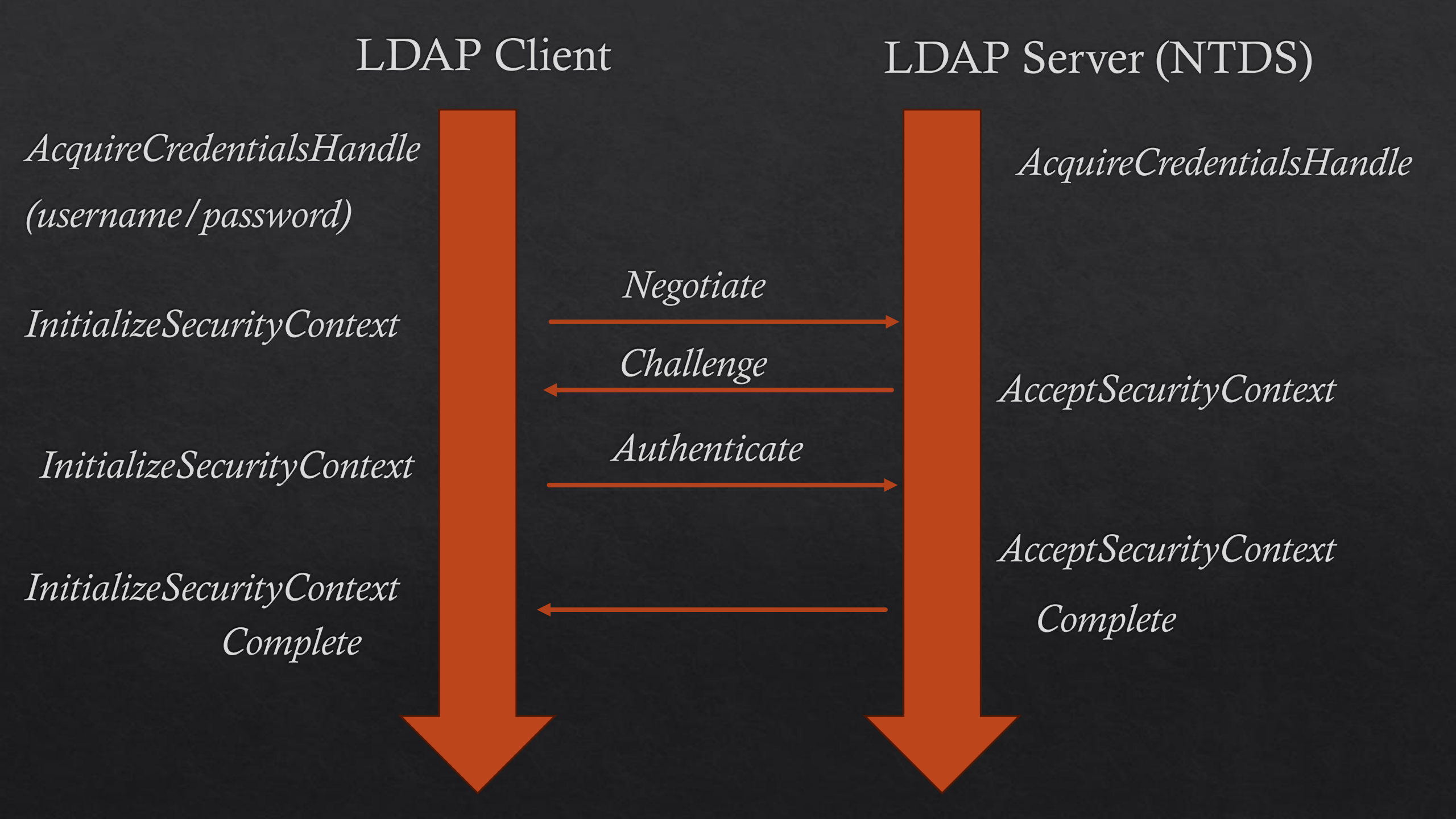
Motivation (Real)



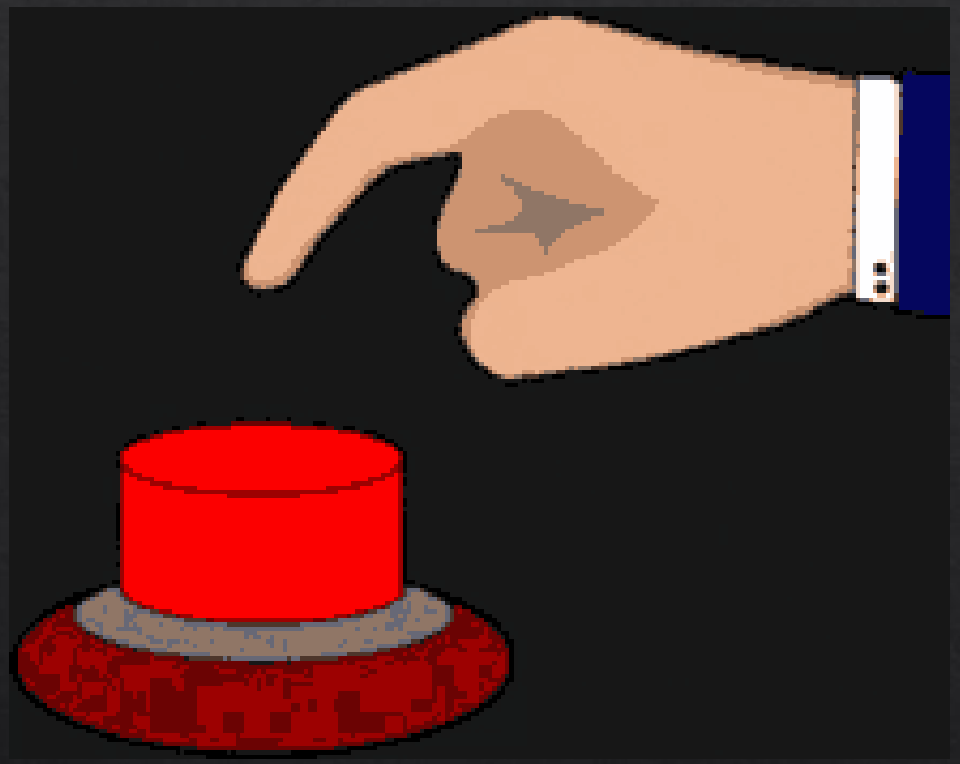
Windows SSPI Architecture

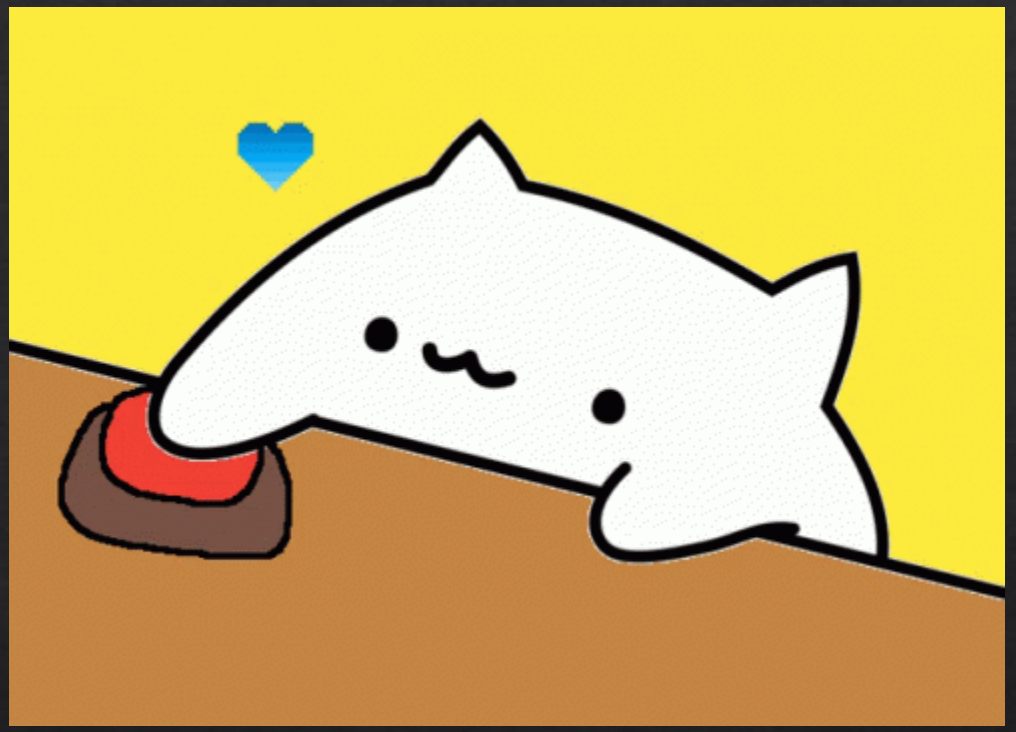
Important SSPI APIs

- AcquireCredentialsHandle
  - Acquires a handle to preexisting credentials (e.g. username/password used in authentication)
- InitializeSecurityContext (ISC)
  - Initiates the client side, outbound security context
- AcceptSecurityContext (ASC)
  - Initiates the server side, inbound security context
- EncryptMessage/DecryptMessage


LDAP Client
LDAP Server (NTDS)
AcquireCredentialsHandle
(username / password)
AcquireCredentialsHandle
Negotiate
InitializeSecurityContext
Challenge
AcceptSecurityContext
Authenticate
InitializeSecurityContext
AcceptSecurityContext
InitializeSecurityContext
Complete
Complete

Doing Authentication with SSPI is Easy



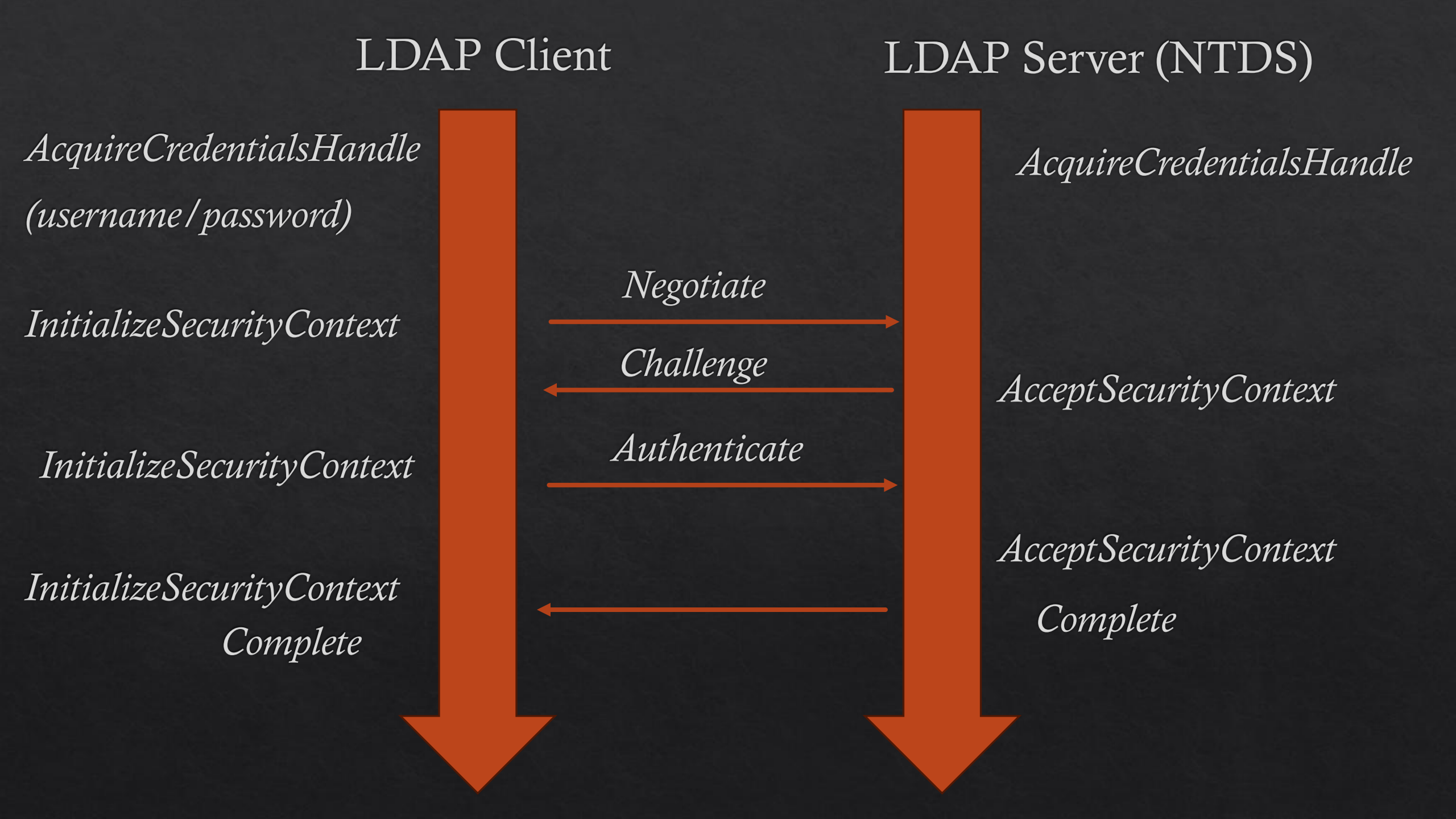


Client: **InitializeSecurityContext** repeatedly until success

Server: **AcceptSecurityContext** repeatedly until success

Under the Hood: Security Packages & Providers

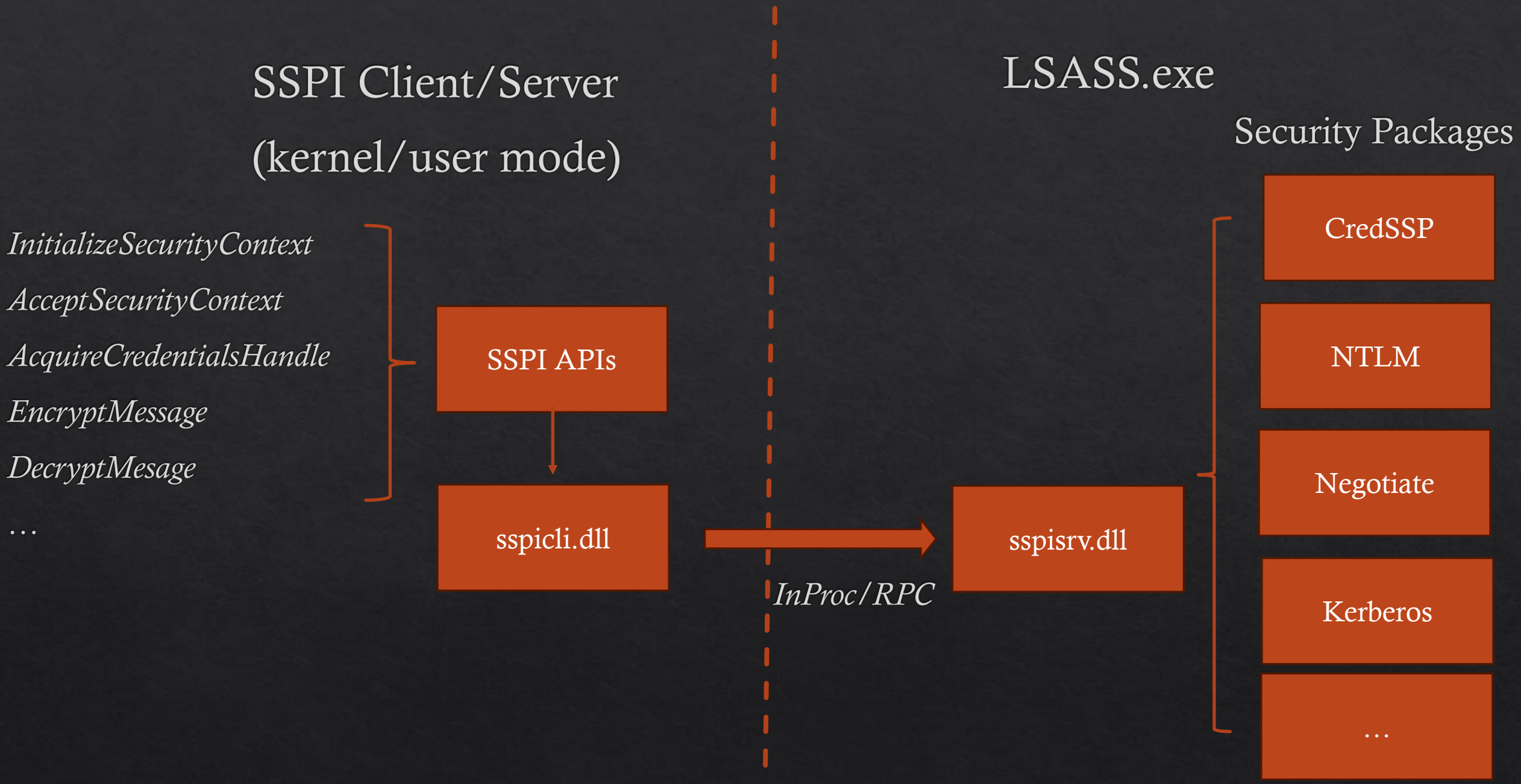
- Security packages
  - The software implementation of a security protocol.
- Security support providers (SSP)
  - Implementation DLLs of security packages
    - NTLM => msv1_0.dll
    - Digest => WDigest.dl
    - Schannel => Schannel.dll
    - CredSSP => CredSSP.dll
    - ….


LDAP Client
LDAP Server (NTDS)
AcquireCredentialsHandle
(username / password)
AcquireCredentialsHandle
Negotiate
InitializeSecurityContext
Challenge
AcceptSecurityContext
Authenticate
InitializeSecurityContext
AcceptSecurityContext
InitializeSecurityContext
Complete
Complete

Local Security Authority Subsystem Service (LSASS)

- Security support providers are loaded by LSASS
  - Sensitive security tasks are performed in LSASS.exe
  - LSASS involves in every Windows authentication

- OOP SSPI client/server calls into LSASS via RPC
  - sspicli.dll: in SSPI client/server processes
  - sspisrv.dll: in lsass.exe


SSPI Client/Server
(kernel/user mode)
LSASS.exe
Security Packages
InitializeSecurityContext
AcceptSecurityContext
AcquireCredentialsHandle
EncryptMessage
DecryptMesage
…
SSPI APIs
sspicli.dll
InProc/RPC
sspisrv.dll
CredSSP
NTLM
Negotiate
Kerberos
…

When a server calls AcceptSecurityContext to authenticate a remote client via NTLM

Call Stack in SSPI server

```
Call Site
ntdll!NtAlpcSendWaitReceivePort+0x14
RPCRT4!LRPC_BASE_CCALL::DoSendReceiv
RPCRT4!LRPC_CCALL::SendReceive+0x51
RPCRT4!NdrpClientCall3+0xdf9
RPCRT4!NdrClientCall3+0xf1
SSPICLI!SspipProcessSecurityContext+
SSPICLI!LsaAcceptSecurityContext+0x2
SSPICLI!AcceptSecurityContext+0x11d
TestAcceptSecurityContext+0x125f
```

Call Stack in LSASS

```
Call Site
msv1_0!SpAcceptLsaModeContext
lsasrv!WLsaAcceptContext+0x6eb
lsasrv!NegAcceptLsaModeContext+0x56a
lsasrv!WLsaAcceptContext+0x6eb
lsasrv!SspiExProcessSecurityContext+
SspiSrv!SspirProcessSecurityContext+
RPCRT4!Invoke+0x73
RPCRT4!Ndr64StubWorker+0xb4a
RPCRT4!NdrServerCallAll+0x40
```

Important Data Structures - Credentials

- Credentials
  - Logon data used by SSP (password/Kerberos tickets/certificate/…)
  - SSPI server/client obtains a credentials handle via AcquireCredentialsHandle API
  - Each SSP has its' own credentials structure

Important Data Structures – Security Context

- Security Context
  - Contains various data regarding the current session (e.g. status/session key/…)
  - SSPI server/client obtains a security context handle via ISC/ASC API
  - Each SSP has its' own security context structure

Context Handle - User

```
typedef struct _SecHandle {
  ULONG_PTR         dwLower;
  ULONG_PTR         dwUpper;
} SecHandle, * PSecHandle;
```

```
0:005> dq rdx
00007ff6`4b645640  000001a9`d285f6d0 0000026d`08b3cea0
```

Security Package

```
dqs 1a9`d285f6d0 +b0 L1
SSPICLI!LsaFunctionTable
```

Remote Context in LSASS.exe

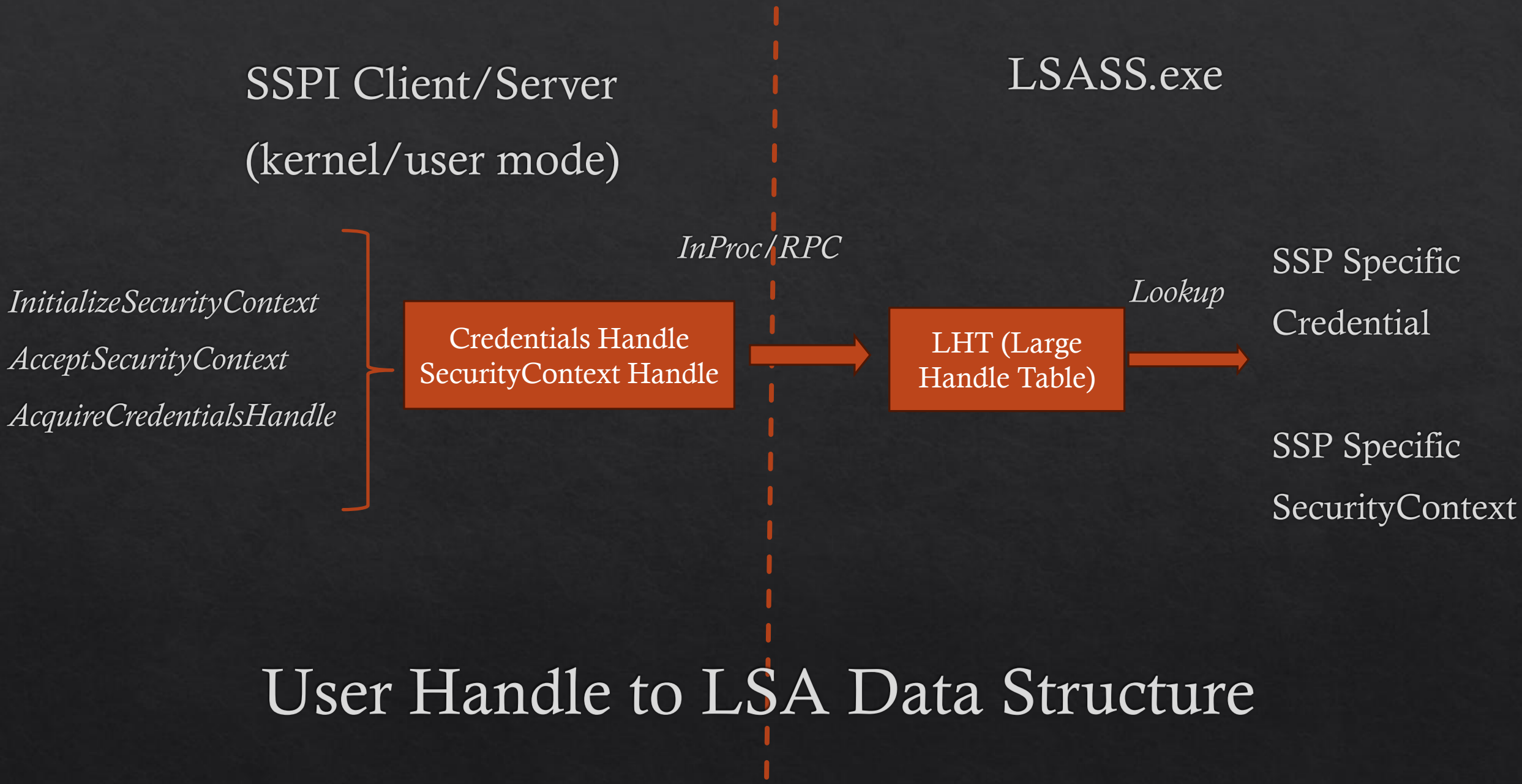
Context Handle - LSASS

```
msv1_0!SpAcceptLsaModeContext:
```

```
0:036> db 26d`08b3cea0
0000026d`08b3cea0  a0 ce b3 08 6d 02 00 00-a0 ce b3 08 6d 02 00 00  ....m.......m...
0000026d`08b3ceb0  43 74 78 41 00 00 00 00-0f 2f 35 01 00 00 00 00  CtxA...../5.....
0000026d`08b3cec0  d2 73 ae 05 6a b3 db 01-e0 93 04 00 01 00 00 00  .s..j...........
0000026d`08b3ced0  35 82 89 e2 1c 00 02 00-00 00 00 00 00 00 00 00  5...............
```

User Mode Context/LSA Mode Context

- User Mode Context
  - User mode security context mapped to SSPI client/server process
  - Contains partial information (e.g. session key)
  - Encrypt/Decrypt/Partial Attributes Get/Set
- LSA Mode Context
  - Full security context stored in LSASS
  - ISC/ASC/Attributes Get/Set



User Handle to LSA Data Structure

Windows SSPI Bugs

Windows SSPI Bugs

- Normal Bugs
- Race Condition Bugs
- API Incorrect Using Bugs

Normal Bugs

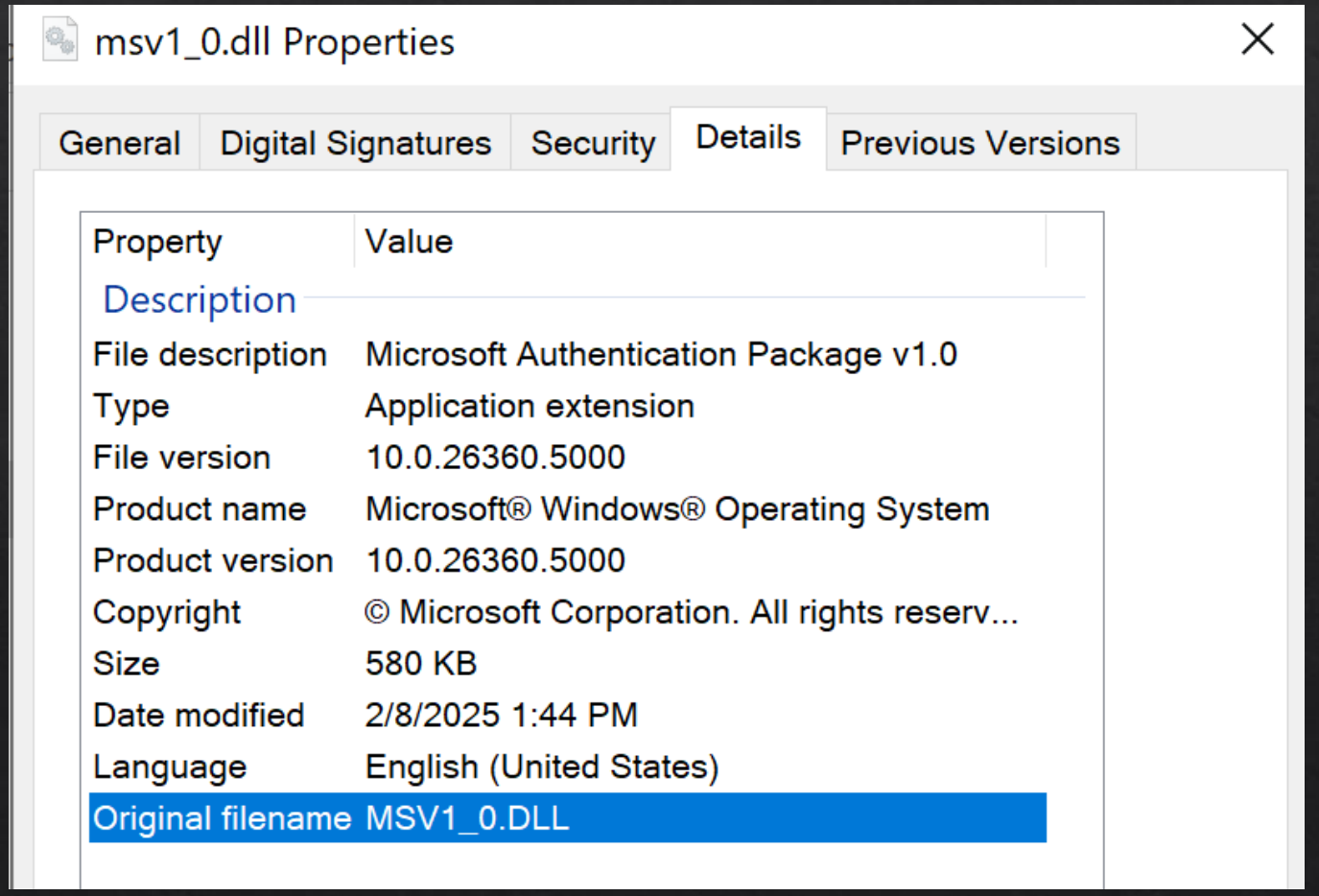
- Trivial bugs that can be found via simply fuzzing & auditing
- Where to look
  - The Security Support Providers (SSP) DLLs
  - SSPI RPC (sspicli.dll/sspisrv.dll/lsasrv.dll)

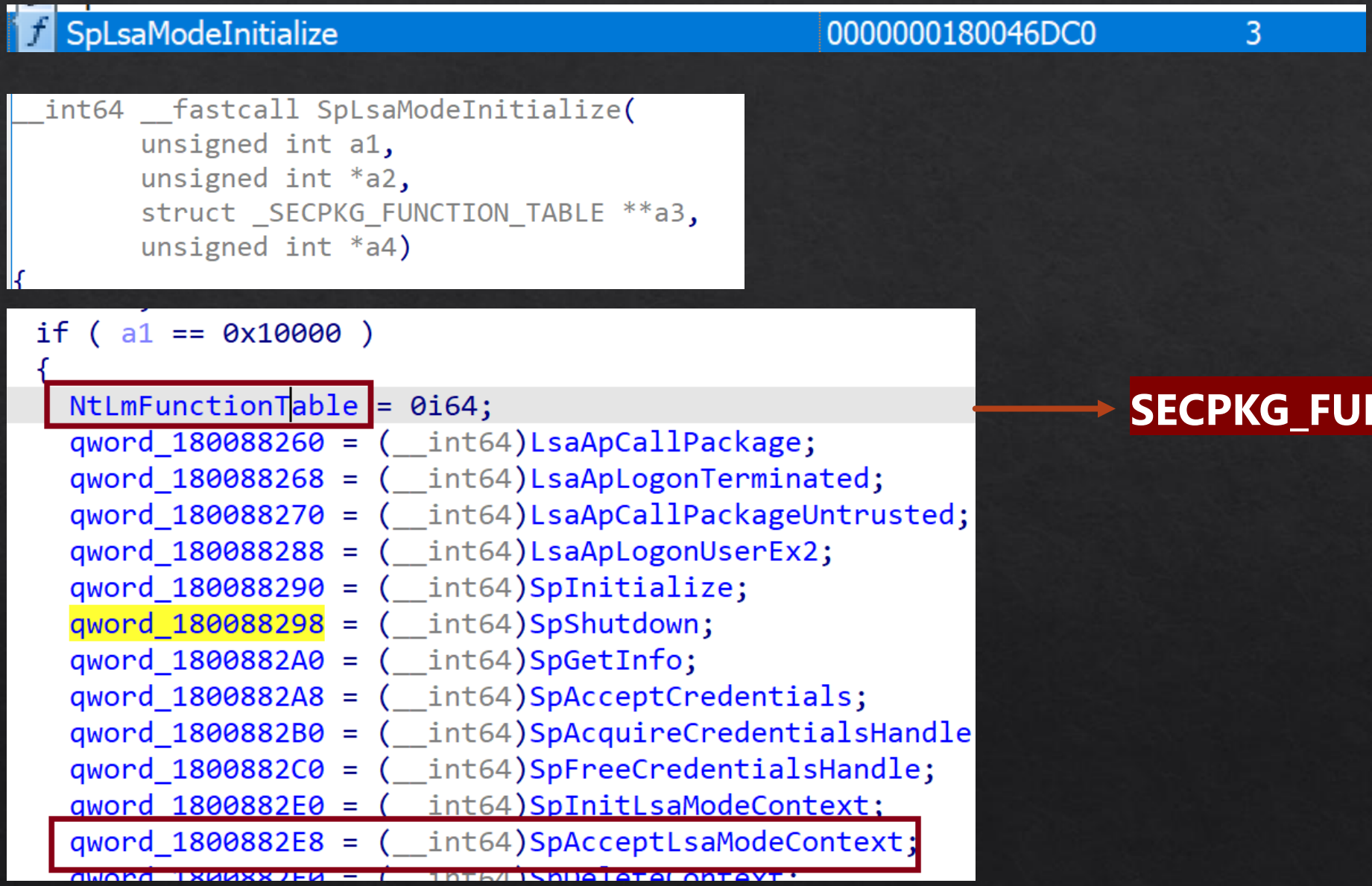
Find SSP Entry Functions

- SSP Export Functions

Name	Address	Ordinal
DllMain	**000000018002F8B0**	**5**
LsaApCallPackage	000000018002BD40	6
LsaApCallPackagePassthrough	0000000180034720	7
LsaApCallPackageUntrusted	0000000180029760	8
LsaApInitializePackage	0000000180034770	9
LsaApLogonTerminated	0000000180006190	10
LsaApLogonUserEx2	000000018001E210	11
Msv1_0ExportSubAuthenticationRoutine	00000001800393F0	12
Msv1_0ExportSubAuthenticationRoutineEx	00000001800394F0	13
Msv1_0SubAuthenticationPresent	0000000180039500	14
MsvGetLogonAttemptCount	0000000180038320	15
MsvIsIpAddressLocal	000000018000F050	16
MsvIsLocalhostAliases	000000018000F3B0	2
MsvSamLogoff	0000000180031770	17
MsvSamValidate	00000001800235B0	18
MsvValidateTarget	0000000180038330	19
SpInitialize	0000000180048F40	1
SpInstanceInit	000000018002C0A0	32
SpLsaModeInitialize	0000000180046DC0	3
SpUserModeInitialize	000000018002ECC0	4
_DllMainCRTStartup	**0000000180031D10**	**[main entry]**

Example: Find Entry for AcceptSecurityContext in NTLM SSP




SpLsaModeInitialize 0000000180046DC0 3
__int64 __fastcall SpLsaModeInitialize(
unsigned int a1,
unsigned int *a2,
struct _SECPKG_FUNCTION_TABLE **a3,
unsigned int *a4)
{
if (a1 == 0x10000)
{
NtLmFunctionTable = 0i64;
qword_180088260 = (__int64)LsaApCallPackage;
qword_180088268 = (__int64)LsaApLogonTerminated;
qword_180088270 = (__int64)LsaApCallPackageUntrusted;
qword_180088288 = (__int64)LsaApLogonUserEx2;
qword_180088290 = (__int64)SpInitialize;
qword_180088298 = (__int64)SpShutdown;
qword_1800882A0 = (__int64)SpGetInfo;
qword_1800882A8 = (__int64)SpAcceptCredentials;
qword_1800882B0 = (__int64)SpAcquireCredentialsHandle
qword_1800882C0 = (__int64)SpFreeCredentialsHandle;
qword_1800882E0 = (__int64)SpInitLsaModeContext;
qword_1800882E8 = (__int64)SpAcceptLsaModeContext;
SECPKG_FUNCTION_TABLE

SECPKG_FUNCTION_TABLE

- https://learn.microsoft.com/en-us/windows/win32/api/ntsecpkg/ns-ntsecpkg-secpkg_function_table

```
typedef struct _SECPKG_FUNCTION_TABLE {
  …
  SpInitLsaModeContextFn              *InitLsaModeContext;
  SpAcceptLsaModeContextFn            *AcceptLsaModeContext;
  SpDeleteContextFn                   *DeleteContext;
  SpApplyControlTokenFn               *ApplyControlToken;
  SpGetUserInfoFn                     *GetUserInfo;
  SpGetExtendedInformationFn          *GetExtendedInformation;
  SpQueryContextAttributesFn          *QueryContextAttributes;
  …
} SECPKG_FUNCTION_TABLE, *PSECPKG_FUNCTION_TABLE;
```

Case Study : CVE-2025-21369

- Classic integer overflow => OOB write
- Digest SSP: WDigest.dll
- Remote, in multi-domain environment
- EoP from domain User => DC

```
wSize = v13 + 2;
v15 = wSize + 48;
Memory = DigestAllocateMemory((unsigned __int16)(wSize + 48));
```



Authentication request:

SSP: Wdigest

Target domain: Domain B



DC of

Domain A

Forword request

Trigger the bug



DC of

Domain B

Case Study : CVE-2025-21295

- Reference counter overflow => UAF
- NEGOEX SSP: Negoexts.dll
- Pre-auth/Remote

```
__int64 __fastcall WSTRetrieveSessionKeys(struct _WST_ACTIVE_ENGINE_CONTEXT *a1)
{
  struct _WST_SSP_PACKAGE *Package; // rsi
  int v4; // ebx
  void *v5; // rcx
  void *v6; // rcx
  void *v7; // rcx
  void *v8; // rcx
  void *Src[2]; // [rsp+38h] [rbp-60h] BYREF
  void *v10[2]; // [rsp+48h] [rbp-50h]

  *(_OWORD *)Src = 0i64;
  *(_OWORD *)v10 = 0i64;
  if ( !a1 )
    return 3221225485i64;
  Package = WSTLocatePackageByAuthScheme((struct _GUID *)((char *)a1 + 24));
  if ( Package )
  {
```

Package structure referenced in WSTLocatePackageByAuthScheme, but never dereferenced







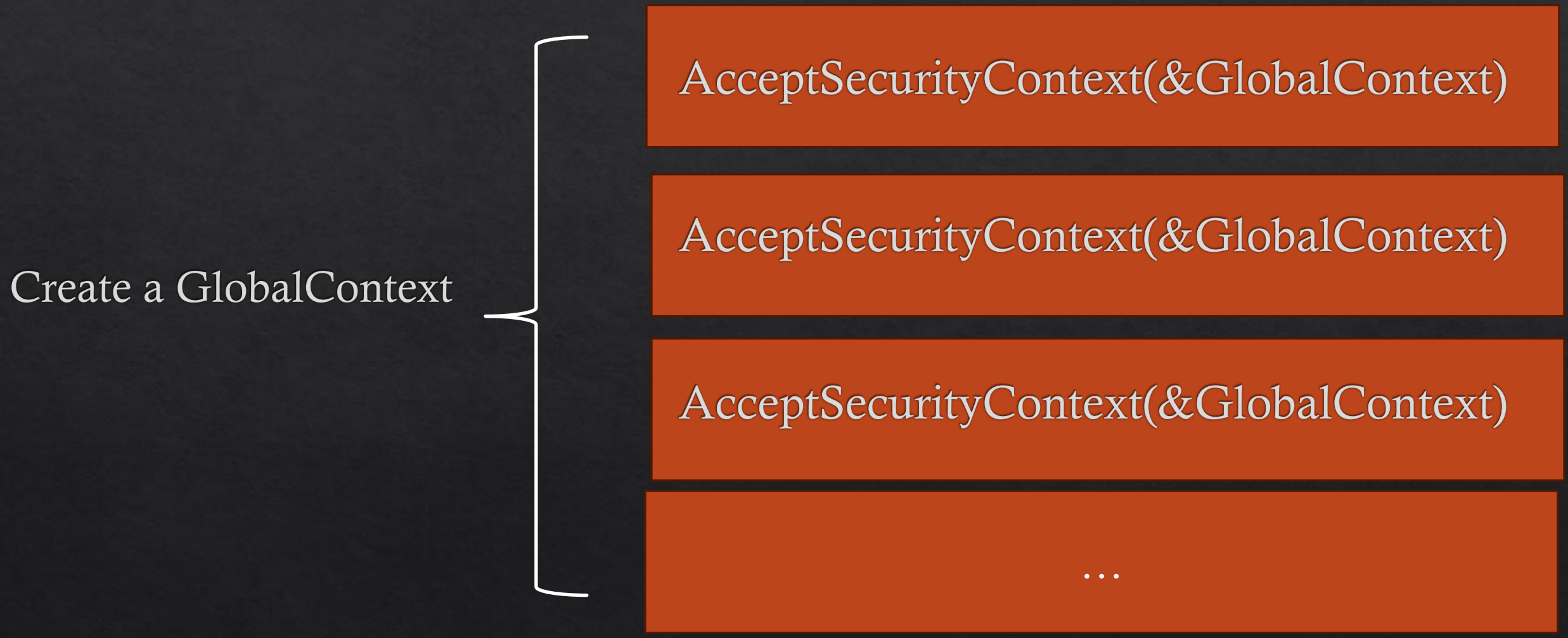
Keep sending NEGOEXTS authentication message until reference counter overflowed and UAF

```
rax=0000000000000000 rbx=0000000000000000 rcx=000000800730ee60
rdx=000000800730efd8 rsi=000000800201cee0 rdi=0000000000000000
rip=00007ffbda3daa4f rsp=0000008002e7ccc0 rbp=0000008002e7cd90
 r8=0000008002e7cdbc  r9=00000000ffffffff r10=00000000fffffef
r11=0000008002e7ca00 r12=0000008007d16ff0 r13=0000000000a11113
r14=0000000000000000 r15=0000000000000000
iopl=0         nv up ei pl zr na po nc
cs=0033  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00010246
negoexts!WSTComputeInitialHelloMessage+0x3df:
00007ffb`da3daa4f 488b8180000000  mov     rax,qword ptr [rcx+80h] ds:00000080`0730eee0=????????????????
```

Race Condition Bugs

- SSPI user calls into LSASS via RPC
- Possible to trigger race condition in lsass.exe?
  - Credentials/Security Context

Quick Testing




THAT'S TOO SIMPLE,
NO WAY TO FIND REAL BUGS
IT DIDN'T
WORK, RIGHT?
imgflip.com

Sometimes Finding Exploitable Bug is Not Difficult ☺

```
cmp     dword ptr [rdx-40h],0ABCDBBBBh ds:0000021c`8ee297c0=????????

         Call Site
04598    verifier!AVrfpDphFindBusyMemoryNoCheck+0x92
0468f    verifier!AVrfpDphFindBusyMemory+0x20
02f7d    verifier!AVrfpDphFindBusyMemoryAndRemoveFromBusyList+0x2b
7a893    verifier!AVrfDebugPageHeapFree+0x8d
e017e    ntdll!RtlDebugFreeHeap+0x37
e6f81    ntdll!RtlpFreeHeap+0x174e
38e56    ntdll!RtlFreeHeap+0x651
```

```
   vmovdqa ymmword ptr [rcx],ymm1 ds:00000236`1b1ae000=??

            Call Site
6cd8c31     ucrtbase!memcpy+0x1e3
6cb2ba0     lsasrv!NegAddFragmentToContext+0x2d
6d0765a     lsasrv!NegAcceptLsaModeContext+0x3f0
6c90195     lsasrv!NegAcceptLsaModeContextEx+0xca
```

```
ucrtbase!memcpy+0x1e7:
00007ffc`33c7dcb7 c5fd7f5120      vmovdqa ymmword ptr [rcx+20h],ymm2 ds:000001bd`c0f1f000=??
0:033> k
 # Child-SP          RetAddr               Call Site
00 00000036`7247cca8 00007ffc`32b87367     ucrtbase!memcpy+0x1e7
01 00000036`7247ccb0 00007ffc`32b901d7     msv1_0!SspMapContext+0x3f7
02 00000036`7247d590 00007ffc`333a0195     msv1_0!SpAcceptLsaModeContext+0x527
```

Most SSPs are Vulnerable to Race Condition

- NTLM (msv1_0.dll)
- Digest (Wdigest.dll)
- Schannel (Schannel.dll)
- Negotiate (lsasrv.dll)
- NEGOEXTS (negoexts.dll)
- CredSSP (credssp.dll)

Microsoft Already Told You About the 0days

> ⚠ **Warning**
>
> Do not use the same context handle in concurrent calls to **AcceptSecurityContext (Schannel)**. The API implementation in the security service providers is not thread-safe.

> ⚠ **Warning**
>
> Do not use the same context handle in concurrent calls to **AcceptSecurityContext (NTLM)**. The API implementation in the security service providers is not thread-safe.

> ⚠ **Warning**
>
> Do not use the same context handle in concurrent calls to **AcceptSecurityContext (Negotiate)**. The API implementation in the security service providers is not thread-safe.

Case Study – AcceptSecurityContext Racing

- NTLM Security Context Domain Name String Double Free

```
Struct NtlmSecurityContext {
…
+ 0x80    UNICODE_STRING DomainName;
…
}
```

AcceptSecurityContext



NTLM Authenticate Message

```
SsprHandleAuthenticateMessage(pContext, …)
{
…
Free(pContext->DomainName);
pContext->DomainName = NewDomainName;
…
}
```

Double free if called by multiple threads

Case Study – InitializeSecurityContext Racing

- Negotiate SSP Fragmentation Buffer OOB Read/Write
- The negotiate protocol supports message fragmentation
  - Both Server/Client side



First InitializeSecurityContext/AcceptSecurityContext



NegCreateContextFromFragment



```
Struct FragmentBuffer
{
DWORD cbTotal;
DWORD cbCurrent;
PVOID   pData;
}
```

Subsequent InitializeSecurityContext/AcceptSecurityContext



NegAddFragmentToContext



```
__int64 __fastcall NegAddFragmentToContext(struct
_NEG_CONTEXT *a1, struct _SecBuffer *pInput)
{
//Boundary check
  if ( pInput->cbBuffer > pFragBuf->cbTotal – pFragBuf->cbCurrent)
    return 2148074240i64;
  memcpy_0(pFragBuf->pData, pInput->pvBuffer, pInput->cbBuffer);
  pFragBuf->cbCurrent += pInput->cbBuffer;
```

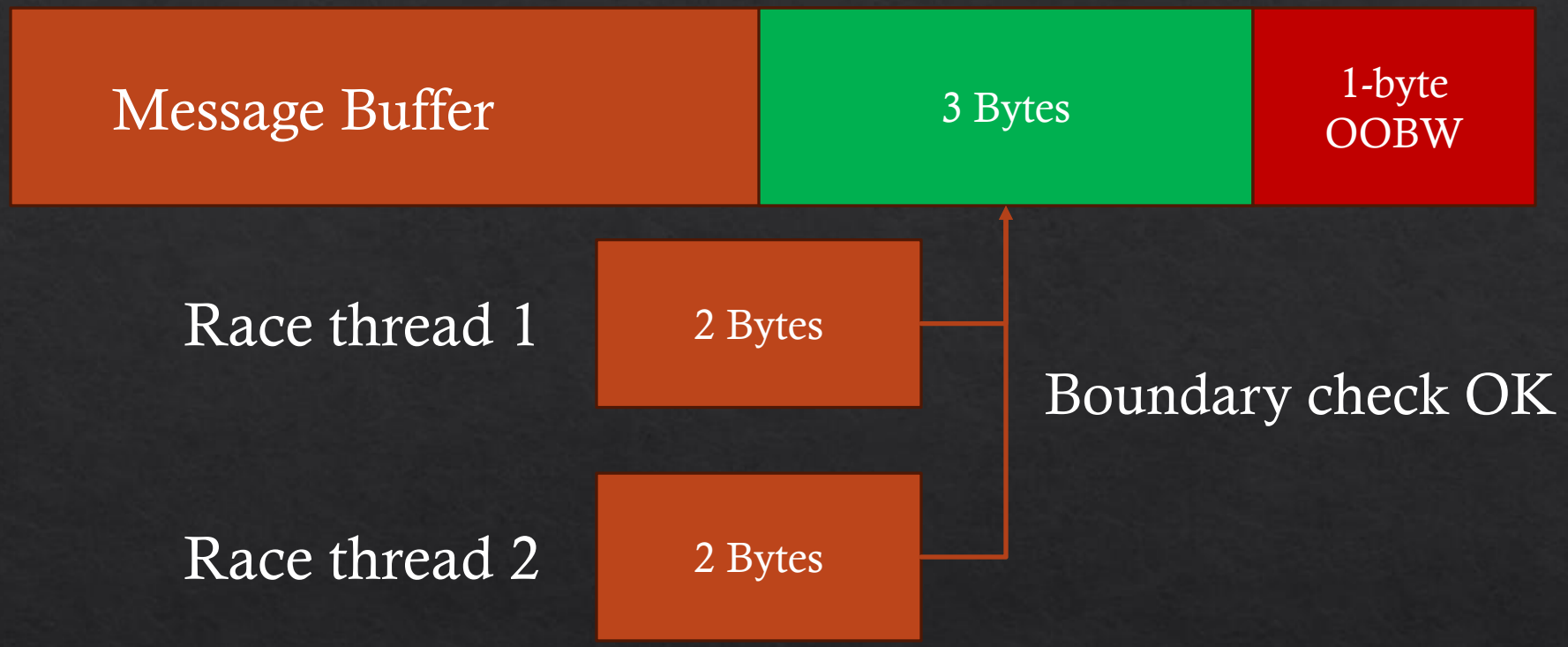
NegAddFragmentToContext

(1) **Check:** NewFrag.Size <= MsgBuffer.RemainSize

(2) **Copy:** memcpy(MsgBuffer, NewFrag, NewFrag.Size)

(3) **Update:** MsgBuffer.RemainSize -= NewFrag.Size

Race Window


Message Buffer
3 Bytes
1-byte
OOBW
Race thread 1
2 Bytes
Race thread 2
2 Bytes
Boundary check OK

OOB Write Primitive

```
__int64 __fastcall NegAddFragmentToContext(struct
_NEG_CONTEXT *a1, struct _SecBuffer *pInput)
{
//Race condition makes cbCurrent > cbTotal, the boundary check
always passed
  if ( pInput->cbBuffer > pFragBuf->cbTotal – pFragBuf->cbCurrent)
    return 2148074240i64;
//Arbitrary write behind the data buffer
  memcpy_0(pFragBuf->pData, pInput->pvBuffer, pInput->cbBuffer);
```

OOB Read Primitive

Can read from fragmentation buffer by calling InitializeSecurityContext with an empty InputBuffer

…

```
        cbBuffer = pOutput->cbBuffer;
if ( pOutput->cbBuffer >= > pFragBuf->cbTotal – pFragBuf->cbCurrent)
        cbBuffer = > pFragBuf->cbTotal – pFragBuf->cbCurrent;
// Read arbitrary size behind the data buffer
        memcpy_0(pOutput_1->pvBuffer, pFragBuf->pData, cbBuffer)
```

Exploit

- We have a heap buffer with controlled size in lsass.exe
- We can read/write arbitrary size data after this buffer
- We can do the read/write for arbitrary times
- Should be trivial to finish the exp

April Patch

- We discovered some SSPI race condition bugs in different SSPs' AcceptSecurityContext/InitializeSecurityContext and reported to MS
- MS released a defense-in-depth patch in April
- Now a LSASS session can not enter AcceptSecurityContext/InitializeSecurityContext in multiple threads at the same time
- But that's not the end of the story☺

Case Study – SetContextAttributes Racing

- Schannel CSsl3TlsContext SetKeyingMaterialInfo Double

```
__int64 __fastcall CSsl3TlsContext::SetKeyingMaterialInfo(#266 *this,
void *a2, unsigned int a3)
{
...
    SPExternalFree(*((void **)this + 320));
...
 }
```

Double free if called by multiple threads

Case Study – ApplyControlToken Racing

- Schannel ApplyControlToken UAF

```
__int64 __fastcall SpApplyControlToken(__int64 phContext, __int64
pInput)
{
...
*(_DWORD *)(pSslContext + 92) = v7;
 }
```

Another thread can free pSslContext causing UAF write

Case Study – CVE-2025-21294 Digest SSP Race Condition（Pre-auth/Remote）



Nonce

- Each Digest security context has a unique nonce
  - Contains 16-bytes random value
  - Initialized by server when context is created



Nonce => Context

- What if multiple requests use the same nonce?



Digest Authentication: *Nonce 1*



Digest Authentication: *Nonce 1*



Digest Authentication: *Nonce 1*

```
CtxtHandlerReferenceToPtr(&pNonceString)
```

=> Same Context!!!

Digest Context Race Condition

- Triggered by remote unauthenticated attacker sending multiple Digest auth requests with same nonce
- Multi-threading access to same security context
  - Resulting in UAF/Double free/OOB Write/…

```
(354.1e54): Access violation - code c0000005 (first/second chance not available)
For analysis of this file, run !analyze -v
rpcrt4!NDRCContextBinding+0x4:
00007ffe`162d2ae4 81790898badcfe  cmp     dword ptr [rcx+8],0FEDCBA98h ds:000002d0`9203bfe8=????????
```

SSPI API Misusing Bugs

- SSPI APIs are used everywhere by Microsoft & Third-party programs for security related functions
  - Make sure to check whether your target uses SSPI
- Using SSPI in an incorrect way can cause serious security vulnerabilities

Case Study – CVE-2024-49127

- Windows Lightweight Directory Access Protocol (LDAP) Remote Code Execution
  - Remote/Pre-auth
  - Enabled by default in Windows DC
  - SSL security context race condition UAF



StartTLS

Normal Request

StopTls

TLS Establishment

AcceptSecurityContext

SSL Security Context

DecryptMessage

EncryptMessage

DeleteSecurityContext

while being used in EncryptMessage

LDAP Server

- Root Cause: LDAP server failed to protect multi-threading SSPI calls

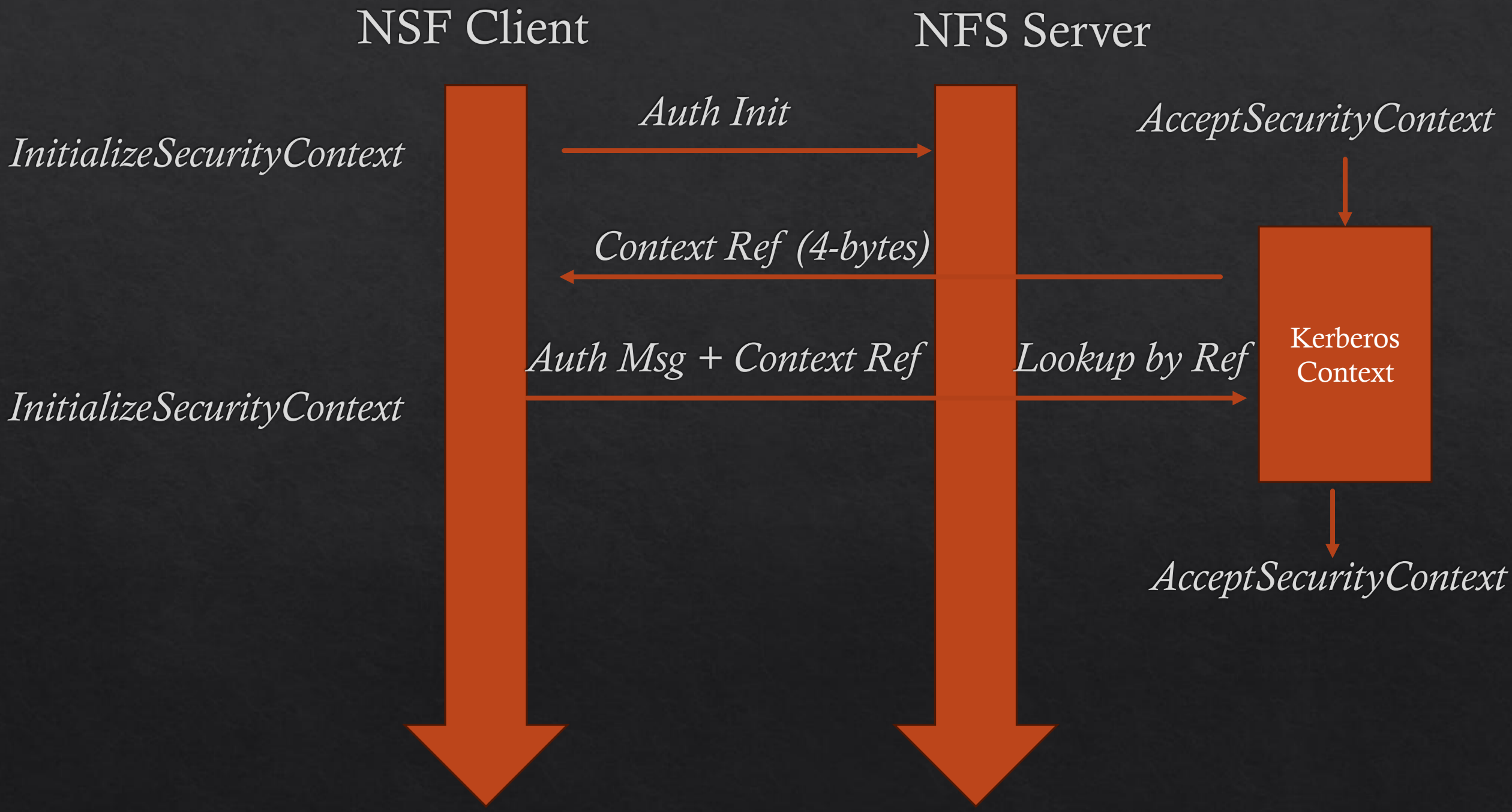
```
schannel!CEvent::ResetEvent+0x4:
00007ffe`f808d664 488b4908        mov     rcx,qword ptr [rcx+8] ds:000001f4`b8371eb0=????????????????
0:036> k
 # Child-SP          RetAddr               Call Site
00 00000064`fe07f170 00007ffe`f806830f     schannel!CEvent::ResetEvent+0x4
01 00000064`fe07f1a0 00007ffe`f82f7da6     schannel!SpSealMessage+0x8f
02 00000064`fe07f1f0 00007ffe`f82f2d17     sspicli!LsaSealMessage+0x226
```

- Combine with another LDAP bug of mine (named LDAPNightmare by a security company) for a possible pre-auth RCE chain

- https://windowsforum.com/threads/cve-2024-49112-critical-ldap-vulnerability-and-the-ldapnightmare-exploit.348822/

Case Study – CVE-???

- Windows Network File System AcceptSecurityContext race condition
  - Remote/Pre-auth
  - Root Cause: the server failed to protect a Kerberos security context from race condition access


NSF Client
NFS Server
Auth Init
InitializeSecurityContext
AcceptSecurityContext
Context Ref (4-bytes)
Kerberos Context
Auth Msg + Context Ref
Lookup by Ref
InitializeSecurityContext
AcceptSecurityContext

Context Reference => Kerberos Context

- NFS server lookups the Kerberos context by the 4-bytes context reference from the remote request







Same Context Reference

Same Context Reference

Same Context Reference

OncRpcSepInboundQueryAndRefGssCtx

AcceptSecurityContext(SameKerberosContext)

Conclusion

- Windows SSPI architecture
- SSPI bug types & Case study
- Keep an eye on multi-threading race condition SSPI issues
- Some interesting bugs still not patched so maybe next time

Thank you!

